

HW2 (PL/0 Lexical Analyzer)

COP 3402: Systems Software

Instructor: Jie Lin, Ph.D.

Due date: **See Webcourses for the posted due date and time.**

Important: This handout is the primary written specification for the homework and shall be read together with course lectures, demonstrations, Webcourses announcements, and any written clarifications issued by the instructor.

If any provision of this handout is unclear, incomplete, or reasonably subject to more than one interpretation, students should request clarification from the instructor before relying on a particular interpretation.

Any clarification or supplementary direction issued by the instructor in lecture, through Webcourses, or in written course communication shall govern the interpretation and administration of this homework.

4.1 The Parts of the Work

1. Read the source file named on the command line and print it back.
2. Recognize identifiers, numbers, reserved words, operators and punctuation, and give each one its token code.
3. Skip comments and whitespace without letting either break a token apart.
4. Build a name table that records each identifier once, with the line and column where it was first seen.
5. Print the lexeme table, the name table and the token list, and write `tokens.txt` and `nametable.txt`.
6. Detect the eleven lexical errors, report the first one with its line and column, and exit with a non-zero status.

4.2 Why the Specification Reads the Way It Does

Why the scanner stops at the first error. A scanner that keeps going has to decide how much of the bad text to throw away, and every such decision is a guess about what the programmer meant. Stopping removes the guess. It also makes the exit status a fact rather than a summary: zero means the whole file scanned, non-zero means it did not.

Why some reserved words appear in no program you will ever write. This language reserves `do`, `od` and `odd`. No statement or expression in it uses them. Your scanner recognizes all three anyway, because deciding whether a word is meaningful where it sits is not the scanner's job. The scanner's job is the vocabulary. What the vocabulary is allowed to spell is a separate question, answered by a separate stage, and one that cannot be answered at all if the vocabulary is wrong. Leave `odd` out of your table and it comes back as an identifier, which is a different kind of thing entirely.

Why the name table stores an index instead of the spelling. An identifier that appears twenty times is one name. Storing it once and referring to it by a number is called **interning**, and it is what `tokens.txt` does: the stream carries 1 0, never 1 **counter**. It is also why the two files have to be read together, which is the point. A real compiler extends exactly this table with each name's kind, level and address, and every later stage reaches a name through its index rather than through its letters.

Why the limits are counted the way a reader counts. An identifier may be twelve characters. That is twelve characters you can see, not eleven characters and a terminator. If you declare an array for one, it needs thirteen bytes.

4.3 What Is Graded

There are one hundred eighty cases and there are no hidden cases. Every input, every expected output and every point value is in your hands, in the test bundle, before you write a line. Ninety points come from what your program prints and ten from the two files it writes. Table 1 gives the split.

Category	Case prefix	Cases	Points
Lexical errors	e	60	25
Operators and longest match	o	30	20
Comments	c	25	11
Scanning basics	s	12	11
Reserved words against identifiers	w	18	10
Whitespace and layout	p	16	9
Name table	n	14	9
Argument and file contract	t	5	5
Total		180	100

Table 1: Points by category, standard output and written files together. Inside a category every case is worth exactly the same for its standard output; ten cases, one in each category but `t`, also carry one point each for the two written files, which is where the ten file points live. The weights follow difficulty. The operators carry the most per case because choosing between a one-character and a two-character token is the decision this homework is about, and a scanner that chooses wrong produces a token list that is wrong from that point to the end of the file. The lexical errors carry the largest total because there are eleven of them and each one is a different behavior.

4.4 What You Need Before You Start

An account on Eustis, a way to copy files to it, and the ability to compile and run a C program there. Nothing else is assumed. This handout states the whole specification, and the test bundle ships the cases and the script that scores them.

4.5 Permitted Language

Write your program in C. No other language is accepted, and no starter code is provided. The one file you write is `lex.c`.

5 Command Line

Your program takes **exactly one argument** after the program name: the path to the PL/0 source file to scan.

```
./lex <input file>
```

Check the argument count before you touch anything else. A program that reads `argv[1]` when there is no `argv[1]` does not print a helpful message; it crashes, or it reads whatever happened to be in memory, and the case that catches it fails for a reason its output does not show. Validate the count, then open the file, then scan.

```
argc == 2
```

The one accepted form. `argv[1]` is the path to the source file.

```
argc != 2
```

Print the usage message below and exit with a non-zero status. Nothing is scanned and neither output file is written.

5.1 The Two Messages

Five cases compare these two messages word for word. They are not free text. Print them exactly as written here, to either stream.

A wrong argument count

```
Usage: ./lex <input file>
```

The file cannot be opened

```
Error:  unable to open input file 'PATH', with the path exactly as it was given
on the command line in place of PATH, inside single quotes.
```

Printing `argv[0]` where the usage message reads `./lex` produces the same line, because that is how the autograder invokes your program.

Never prompt for input. Your program reads the file named on the command line and nothing else. It must not read standard input, and it must not ask the user a question. A program that waits for input waits until the five-second limit runs out, and a case that times out scores nothing.

If your program prints the four output sections when it is given no argument, you checked `argc` too late. Five cases exercise this: no argument, two arguments, three arguments, a path that does not exist, and a path whose directory does not exist. All five require a non-zero exit status and the matching message, and nothing else. That is five points for four lines of code at the top of `main`.

6 Input Files

The input is a plain text file holding a PL/0 source program. Its path is `argv[1]`.

Read the whole file, however long it is. There is no size limit in this specification. One graded case is 5,770 bytes and produces 1,706 tokens and 100 distinct identifiers. A buffer sized for the examples printed in this handout is not a buffer sized for the cases.

Read it byte by byte and stop at the end of the file, not at the first zero byte. A source file may hold any byte at all. One graded case holds a zero byte in the middle of a line, and it is error 10 in Section 14 rather than the end of the program.

Two conditions are settled before scanning begins.

The file cannot be opened

Print the message from Section 5.1 and exit with a non-zero status. Nothing is scanned and neither output file is written.

The file holds no token

A file that is empty, a file holding only whitespace, and a file holding only a comment all produce no token at all. That is error 11 in Section 14, and it is reported at line 1, column 1.

A file whose last line has no newline at the end is an ordinary file, and one graded case is exactly that. It is not an error, and it changes nothing about how the last token is recognized. When you print the source back, end it with a newline anyway, so that the section after it starts on its own line.

If the short cases pass and the long ones stop partway through, your buffers are a fixed size. The failure looks like a correct scanner that gave up: the lexeme table simply ends, and the diff is one long block of missing lines with nothing wrong in the lines above it. Grow the source buffer, the token array and the name table as you fill them, or size them from the file's length.

7 The Token Table

Every token your scanner produces carries one of the codes below. The numbers are a convention of this homework, not a fact about PL/0, and they are the numbers the autograder compares against.

Do not renumber them, add to them or leave one out. The token list is compared value by value, so a scanner with its own numbering fails every case from its first token onward even when its recognition is perfect.

Two codes are the open classes, whose spelling is not fixed in advance:

Code	Name	What it matches
1	identsym	a letter followed by letters and digits
2	numbersym	one or more digits

Table 2: The two open token classes.

Code	Name	Spelling
3	plussym	+
4	minussym	-
5	multsym	*
6	slashsym	/
7	eqsym	==
8	neqsym	!=
9	lessym	<
10	leqsym	<=
11	gtrsym	>
12	geqsym	>=
13	lparentsym	(
14	rparentsym	)
15	commasym	,
16	semicolonsym	;
17	periodsym	.
18	assignsym	=
19	initsym	:=

Table 3: The seventeen special symbols.

Three spellings that look alike mean three different things. == compares, = assigns, and := gives a constant its value where the constant is declared. There is no <> in this language, and not-equal is !=.

Code	Name	Spelling
20	beginsym	begin
21	endsym	end
22	ifsym	if
23	fisym	fi
24	thensym	then
25	whilesym	while
26	elihwsym	elihw
27	dosym	do
28	odsym	od
29	oddsym	odd
30	callsym	call
31	constsym	const
32	varsym	var
33	procsym	procedure
34	writesym	write
35	readsym	read
36	elsesym	else

Table 4: The seventeen reserved words.

There is no code for a bad lexeme. Some scanners reserve one so that scanning can carry on past an error. Yours stops at the first error, so nothing follows it and there is nothing for such a code to hold.

8 Identifiers, Numbers and Reserved Words

An identifier begins with a letter and continues with letters and digits, up to twelve characters. A thirteenth character is error 2.

A number is one or more digits, up to six digits. A seventh digit is error 3, and a leading zero is a digit like any other: 0000001 is seven digits and is too long.

Collect the whole run before you decide what it is. When the next character is a letter, read every letter and digit that follows it, and then look the finished word up in Table 4. When the next character is a digit, read every digit that follows it. The decision comes after the run, never during it.

The reserved word lookup is exact and case sensitive. A word is a reserved word only when the complete run equals one of the seventeen spellings, character for character. `Begin` is an identifier. `beginning` is an identifier. `od` is a reserved word and `odd` is a different reserved word.

8.1 A Worked Example

This line holds six runs of letters and digits. Four of them are the reserved words `od`, `odd`, `do` and `fi`, and the other two are identifiers:

```
od odd odds do dod fi
```

What the scanner makes of it

Lexeme Table:

lexeme	token
od	28
odd	29
odds	1
do	27
dod	1
fi	23

Name Table:

index	name	line	column
0	odds	1	8
1	dod	1	16

Token List:

```
28 29 1 0 27 1 1 23
```

`odds` and `dod` are identifiers because the finished run is not one of the seventeen spellings, even though each one begins with a reserved word. They take name-table indices 0 and 1, in the order they are first seen.

If `odd` and `odds` both come out as token 28 and your Name Table is empty, your reserved word lookup matches prefixes. A lookup that compares only as far as the keyword's own length accepts `od` as a match for every run beginning with those two letters. Against the line above, that scanner prints `28 28 28 27 27 23`: three reserved words where there should be two and an identifier, no identifiers at all, and an empty name table. Compare the finished run against the whole spelling, and compare its length too.

9 Operators and Longest Match

Five operators are two characters long, and each one begins with a character that either is or is not a token on its own. This is the part of the homework that most often goes wrong, and each of its cases is worth the most.

When the longer operator is available, take it. Look at the character after the one you are holding, and decide from the pair.

Holding	Next is =	Next is anything else	Alone it
=	== (7)	= (18)	is a token
<	<= (10)	< (9)	is a token
>	>= (12)	> (11)	is a token
!	!= (8)	error 5	is not a token
:	:= (19)	error 4	is not a token

Table 5: The two-character operators, looked up by the character you are holding.

The last column is the whole difference between the two halves of that table. For =, < and >, failing to find the = still leaves you holding a complete token. For ! and :, failing to find it leaves you holding nothing, and that is an error rather than a shorter token.

The two characters have to be adjacent. Anything at all between them, a space, a tab, a newline or a comment, breaks the pair, and then each half is decided on its own. Section 11 has the rule and what it costs.

Take one character of lookahead and no more. Every decision in that table needs exactly one character beyond the one you are holding, and that character may not be there at all: a file may end in <, which is token 9, or in !, which is error 5. A scanner that consumes the character it peeked at even when the pair did not match loses the token that follows it, and one that reads past the end of the file to look for a pair crashes on cases that are not errors.

9.1 A Worked Example

Four runs of operator characters, with no space inside any of them:

```
a===b  x:==y  a<>b  a!==b
```

What the scanner makes of it

Lexeme Table:

lexeme	token
a	1
==	7
=	18
b	1
x	1
:=	19
=	18
y	1
a	1
<	9
>	11
b	1
a	1
!=	8
=	18
b	1

Token List:

1 0 7 18 1 1 1 2 19 18 1 3 1 0 9 11 1 1 1 0 8 18 1 1

Each run takes the long operator first and leaves what is left to stand on its own. `a===b` is `a`, `==`, `=`, `b`: the pair is taken long, and the single `=` left over is a complete token by itself. `x:=y` is `x`, `:=`, `=`, `y`. `a!==b` is `a`, `!=`, `=`, `b`.

`a<>b` is the run with no long operator in it. `<>` is not an operator in this language, so it is `<` and then `>`, two complete tokens, and neither character is an error.

If your lexeme table shows `<` and `=` on two lines where the expected output shows `<=` on one, you took the shortest match. This is what it looks like in `failed_cases/o04_relational_all/diff.txt`, where the lines beginning `<` are yours:

```
11,12c11
< <                9
< =                18
---
> <=                10
30c29
< 1 0 9 1 1 9 18 1 2 11 1 3 12 1 4
---
> 1 0 9 1 1 10 1 2 11 1 3 12 1 4
```

One extra token at line 11 shifts every value in the token list after it, so the case loses its output points and, where it carries one, its file point as well. Six cases hold a `<=`.

10 Comments

A comment opens with `/*` and closes with `*/`, and everything between them is discarded. A comment is not a token and never appears anywhere in your output. It may span any number of lines.

Nothing inside a comment is examined. Characters that would be errors anywhere else, and bytes that are not ASCII at all, are skipped along with the rest of the text.

Comments do not nest. A second `/*` before the comment closes is error 9.

A comment separates tokens but is not required to. `x/**/=1;` is the four tokens `x`, `=`, `1` and `;`, exactly as `x = 1 ;` is.

Three things that look like comment delimiters are not:

A slash not followed by a star

The division operator, `slashsym` (6). A file may end in one.

A star not followed by a slash

The multiplication operator, `multsym` (5). A file may end in one.

A star and a slash outside a comment

Error 8. A `*/` with no opener closes a comment that was never opened, and it is usually a comment whose opening delimiter was deleted while editing.

A star immediately followed by a slash is error 8 wherever it appears outside a comment, including where you meant to multiply and then divide. In `b*/c` the scanner reports error 8 at the star. Put a space between the two operators and both are recognized.

10.1 Delimiters That Are Only Half There

Both characters of a delimiter have to be adjacent, and the scanner never reuses one. That one rule settles every shape below, and the bundle has cases for all of them.

A slash and a star with anything between them

Not an opener. `a / * b` is division and then multiplication, and a slash ending one line with a star beginning the next is the same two tokens.

A star and a slash with anything between them, inside a comment

Not a close. The comment is still open after `* /` and after a star that ends a line with a slash beginning the next, and it stays open until a real `*/` or the end of the file.

`/*/`

An opener, then a slash that is ordinary comment text. It does not close anything, because the slash the eye pairs with the star has already been consumed as the second character of the opener. A file holding only `/*/` is error 7.

`**/`

An opener's worth of stars is not needed: this is one star of comment text followed by the close.

`/**/`

The shortest comment there is. The opener takes the first two characters and the close takes the other two.

`*/*`

A close with no opener, which is error 8 at the first character. The scanner never gets as far as reading the third character as part of an opener.

Reaching the end of the file inside a comment is error 7, reported at the line and column of the `/*` that opened it rather than at the end of the file, because the opener is what has to be fixed. A comment that ends the file on a star, on a slash, or on `* /` is this error and not a closed comment.

A closed comment leaves no trace behind it. After `/*` one `*/` the scanner is outside a comment again, so a second `/*` that never closes is still error 7, a stray `*/` after it is still error 8, and a `/*` inside a later comment is still error 9. A scanner that sets a flag and forgets to clear it passes the single-comment cases and fails the ones with two.

If a whole run of your program vanishes from the lexeme table, a comment swallowed it. Scanning for the pair `*/` is right; stopping at the first star and assuming the next character closes the comment is not. Given `/* star * inside */ var q;`, a scanner that closes at the first star reads `inside` as an identifier, reaches the real `*/` with no comment open, and stops with error 8 at column 18 after a single token. The rest of the program is never seen.

11 Whitespace, Lines and Columns

Four characters are whitespace: the space, the tab, the carriage return and the newline. Any run of them separates tokens and produces nothing.

11.1 Where Whitespace Matters and Where It Does Not

Whitespace between two tokens is optional when their character classes differ. `var x,y;begin x=1;y=x+2 end.` scans to exactly the same tokens as the same program written with a space at every join. The scanner ends a run when the character class changes, and it does not need a separator to know that `;` is not part of `1`.

Whitespace is required in exactly two places.

Between a number and a letter that follows it

`2end` is a digit run followed by a letter, which is error 6 with the whole run `2end` as the lexeme. `2 end` is the number `2` and the reserved word `end`.

Between two words

`beginend` is one run of letters, so it is one identifier, not `begin` and `end`. `doododd` is one identifier too.

Whitespace inside a two-character operator breaks it into two decisions. `x < = y` is four tokens: `x`, `<` (9), `=` (18), `y`. So is `x <` followed by `= y` on the next line, and so is `x</**/=y`, where a comment sits between the two characters. For `:` and `!` the same separation is an error instead: `k : = 3` is error 4, because the colon is alone.

11.2 Counting Lines and Columns

Every error message names a line and a column, and every name-table entry records where its identifier was first seen. Both use the same rule.

Lines and columns both start at 1. The first character of the file is line 1, column 1.

A newline

Begins a line: add one to the line number and set the column back to 1.

A tab

One column, like any other character. It does not advance to the next multiple of eight.

A carriage return

Whitespace that advances nothing. It adds no column and it begins no line.

Every other character

One column, including a character inside a comment.

The carriage return rule is what makes a file written on Windows report the same columns as the same file written anywhere else. A file with CR LF line endings carries a carriage return before every newline. Count it as a column and every column you report after the first line is one too large, which fails every case that carries a position, including cases with no error in them at all.

A token's position is the position of its first character. For an error covering a run of characters, it is the position of the first character of the run.

If your columns are right on line 1 and one too large everywhere after it, you are counting the carriage return. Sixteen cases are about layout alone, and several of them use CR LF endings, carriage returns with no newline at all, or the two mixed in one file. A scanner that treats CR as an ordinary character passes every case whose file uses bare newlines and fails every case that does not, which looks like a position bug with no pattern until you look at the bytes.

12 The Name Table

The name table holds each identifier once. The first time an identifier is seen it is added, with the next free index starting at 0, and with the line and column where it was seen. Every later occurrence of the same spelling uses the index it already has.

Only identifiers go in the name table. Reserved words do not, and neither do numbers, operators or punctuation. A program with no identifiers has an empty name table, which prints as the heading and its column titles with nothing under them.

The comparison is exact and case sensitive, the same comparison the reserved word lookup uses. Total, total, TOTAL and ToTaL are four names with four indices.

12.1 A Worked Example

```
var x, y;  
begin  
  y = 3;  
  x = y + x  
end.
```

The name table and token list for that program

Name Table:

index	name	line	column
0	x	1	5
1	y	1	8

Token List:

32 1 0 15 1 1 16 20 1 1 18 2 3 16 1 0 18 1 1 3 1 0 21 17

Two names, six occurrences. `x` is first seen at line 1, column 5, and takes index 0. `y` is first seen at line 1, column 8, and takes index 1. Each appears twice more, and neither is added again: in the token list the four later occurrences read 1 1, 1 0, 1 1, 1 0, in source order.

If your name table has more rows than the program has distinct identifiers, you are adding every occurrence instead of searching first. Against the program above, that scanner prints six rows, `x` at indices 0, 3 and 5 and `y` at 1, 2 and 4, and a token list in which the same variable carries a different number every time it is used. The table alone still looks plausible; the token list is what gives it away. Search the table before you add.

13 Required Output

13.1 The Four Sections

Your program prints exactly four sections to standard output, in this order: **Source Program**, **Lexeme Table**, **Name Table**, **Token List**. Each heading ends with a colon and is followed by a blank line, and a blank line separates each section from the next.

Source Program:

The contents of the input file, exactly as read.

Lexeme Table:

A header line reading `lexeme` and `token`, then one line per token in the order encountered: the spelling, then its code.

Name Table:

A header line reading `index`, `name`, `line`, `column`, then one line per identifier in index order.

Token List:

The codes in order, separated by spaces, on one line. An identifier prints its code and

then its name-table index. A number prints its code and then its digits. Everything else prints its code alone.

Your columns do not have to line up with the reference's. The autograder ignores blank lines and any difference in spaces and tabs, so the width you give a column is yours to choose. What it compares is the text, its order, and where your line breaks fall.

13.2 A Complete Example

This is case `s01_minimal` from the test bundle. The input file:

```
var x;  
begin  
  x = 5  
end.
```

Standard output for `s01_minimal`

Source Program:

```
var x;  
begin  
  x = 5  
end.
```

Lexeme Table:

lexeme	token
var	32
x	1
;	16
begin	20
x	1
=	18
5	2
end	21
.	17

Name Table:

index	name	line	column
0	x	1	5

Token List:

```
32 1 0 16 20 1 0 18 2 5 21 17
```

Read the token list against the lexeme table above it. 32 is var. 1 0 is the identifier at name-table index 0, which is x. 16 is the semicolon, 20 is begin, 1 0 is x again with the same index, 18 is the assignment =, 2 5 is the number 5, 21 is end, and 17 is the period.

13.3 The Two Files

Your program writes two files into the directory it is run from, and it writes them whether or not it found an error. On an error they hold what was scanned before the error, the same as the printed sections do.

tokens.txt

One token per line. A symbol or reserved word is its code alone. An identifier is its code, a space, and its name-table index. A number is its code, a space, and its digits.

nametable.txt

One identifier per line: its index, its spelling, the line it was first seen on, and the column, separated by single spaces.

tokens.txt and nametable.txt for s01_minimal

tokens.txt	nametable.txt
32	0 x 1 5
1 0	
16	
20	
1 0	
18	
2 5	
21	
17	

The two files are read together. tokens.txt names identifiers only by index, so it cannot be read without the name table beside it. That is the point of storing a name once.

The file names are exact and case sensitive. Tokens.txt and token.txt are files the autograder does not find, and a case that carries file points and finds no file earns none of them.

14 Errors and Graceful Exit

Your scanner stops at the first lexical error. It prints everything it scanned before the error, then the error, then exits with a non-zero status. Nothing after the error is scanned, and nothing after it appears anywhere in your output. A file with four bad characters in it reports the first one and says nothing about the other three.

The error line is printed after the token list, in this form:

Error <n> at line <L>, column <C>: <message>
--

There are eleven errors. The message text is compared, so print it exactly as given here.

Error 1

invalid character 'c', with the character in place of c. A printable ASCII character that is not part of this language: ##, \$, @, ?, _, ', ' , a bracket, a brace, a backslash.

Error 2

identifier too long 'lexeme', with the whole run in place of lexeme. A letter-led run longer than twelve characters.

Error 3

number too long 'lexeme'. A digit run longer than six digits.

Error 4

':' must be followed by '='. A colon that is not part of : =.

Error 5

!' must be followed by '='. An exclamation mark that is not part of !=.

Error 6

number followed by a letter 'lexeme', with the whole alphanumeric run in place of lexeme. A digit run with a letter immediately after it, such as 123abc.

Error 7

comment is not closed before end of file. Reported at the position of the /* that opened it.

Error 8

/' without a matching '/'.

Error 9

/*' inside a comment.

Error 10

byte 0xHH is not part of this language, with two upper-case hexadecimal digits in place of HH. Any byte outside the printable ASCII range 0x20 to 0x7E that is not one of the four whitespace characters.

Error 11

no tokens in the source program. Reported at line 1, column 1.

Error 1 and error 10 divide the same job by whether the byte can be shown. Error 1 is for a character you can see and type: a punctuation mark this language has no use for. Error 10 is for everything else, and the cases include a zero byte, a delete (0x7F), a control byte (0x1F), a byte with its high bit set (0x80), and the three-byte curly quotation marks a word processor produces.

Error 10 never prints the byte itself. A smart quote, an en-dash or a non-breaking space pasted out of a word processor or a PDF is two or three bytes, none of which is a character a terminal can show, and a control byte is worse. Printing the value gives the reader something to act on and keeps the output readable; printing the byte puts an unreadable character in the middle of a diff. A tab inside a line is ordinary whitespace and is not this error.

Error 6 is reported instead of error 3 when both apply. In 1234567abc the digit run is both too long and followed by a letter. The letter is what makes the token's kind ambiguous, and that is the more fundamental problem, so it is the one reported, with the whole alphanumeric run as the lexeme. A digit run followed by an underscore is not this error: the underscore is not a letter, so the number ends and the underscore is error 1.

How much input an error names depends on the error. Errors 2, 3 and 6 name the whole run, because the whole run was meant as one thing. The others name a single character or a two-character delimiter.

14.1 A Worked Example

This is case `e15_valid_prefix_then_error`. The input file:

```
const k := 3;
var total;
begin
  total = k + 1;
  total = total @ 2
end.
```

Standard output for e15_valid_prefix_then_error

Source Program:

```
const k := 3;
var total;
begin
    total = k + 1;
    total = total @ 2
end.
```

Lexeme Table:

lexeme	token
const	31
k	1
:=	19
3	2
;	16
var	32
total	1
;	16
begin	20
total	1
=	18
k	1
+	3
1	2
;	16
total	1
=	18
total	1

Name Table:

index	name	line	column
0	k	1	7
1	total	2	5

Token List:

31 1 0 19 2 3 16 32 1 1 16 20 1 1 18 1 0 3 2 1 16 1 1 18 1 1

Error 1 at line 5, column 17: invalid character '@'

Everything before the @ is there and everything after it is not. The 2 later on line 5, and the end and the period on line 6, never appear, because the scanner never reached

them. `tokens.txt` and `nametable.txt` for this case hold the same eighteen tokens and the same two names.

14.2 The Exit Status

0 The whole file scanned with no lexical error.

non-zero Anything else: a lexical error, a wrong argument count, or a file that could not be opened.

The autograder checks the status of every case, and sixty-five cases require a non-zero one. Check yours after a run:

```
./lex test_cases/inputs/e15_valid_prefix_then_error.txt  
echo $?
```

If a case fails and your output looks identical to the expected file, check your exit status. A scanner that reports the error correctly and then returns 0 from `main` fails all sixty error cases and all five argument cases, twenty-nine points, without a single wrong character anywhere in its output. The last line of `main` decides this.

15 Compile and Run

Compile on Eustis with exactly this command:

Compiling

```
gcc -Wall -Wextra -std=c11 -O2 lex.c -o lex
```

Compiler warnings do not fail the build and cost you nothing. They are worth reading, because most of them are a bug the compiler noticed before you did, but a program that warns and passes the cases scores full marks.

Run it on one of the bundled cases:

```
./lex test_cases/inputs/s01_minimal.txt
```

16 Code Requirements

16.1 The File Name

Your source file must be named `lex.c`.

The grader does not rename your file and does not guess what you meant: the name is how a submission is identified, compiled and tested. Section 20.3 states what a wrong one costs.

The comparison is exact and case sensitive: any other spelling, any change of case, a different base name, and any suffix such as `final` or `v2` are rejected.

You will find out on Eustis long before you submit. The autograder enforces the same rule, and refuses to compile a wrongly named file:

FAIL: Wrong file name: <the name you used>

Rename the file and run it again.

16.2 The Header Comment

Every submission begins with the header comment below, with **Author(s):** filled in. It ships in the test bundle as `header_comment.txt`, so copy it from there rather than retyping it. Section 20.3 states what a missing header, or an author line left at the placeholder, costs.

The required header comment

```
/*
Homework:
lex - HW2 PL/0 lexical analyzer

Author(s): <Your Name Here>

Language: C only

To Compile:
gcc -Wall -Wextra -std=c11 -O2 lex.c -o lex

To Execute (on Eustis):
./lex <input_file>

where:
<input_file> is the path to a text file holding a PL/0 source program

Notes:
- Implements the lexical analyzer described in the homework
  instructions.
```

The required header comment (continued)

- Prints four sections to standard output: Source Program, Lexeme Table, Name Table and Token List.
- Writes two files into the working directory: tokens.txt and nametable.txt.
- Stops at the first lexical error, prints everything scanned before it, reports the error with its line and column, and exits with a non-zero status.
- Exits with status 0 when the whole program scans without an error.
- Tested on Eustis.

Class: COP 3402 - Systems Software

Instructor: Jie Lin, Ph.D.

Due Date: See Webcourses

*/

16.3 Other Rules

- **One source file.** Everything you write goes in `lex.c`.
- **No interactive input.** Your program reads the file named on the command line and nothing else.
- **Standard output carries the four sections and, on an error, the error line.** Debugging output left in a submission is compared along with everything else and fails every case it appears in.
- **Comment your code.** The scanning loop, the reserved word lookup and the comment skipper each carry a comment saying what they do.

17 The Test Bundle and the Autograder

17.1 What You Are Given

Download `lex_test.zip` from Webcourses. It unpacks to one folder, `lex_test`:

What unzipping gives you

<code>lex_test/</code>	
<code>test_lex.sh</code>	the autograder
<code>header_comment.txt</code>	the header block this handout requires
<code>test_cases/cases.txt</code>	every case, with what each one is worth
<code>test_cases/purposes.txt</code>	what scenario each case covers, one line each
<code>test_cases/inputs/</code>	the PL/0 programs the cases scan
<code>expected_results/</code>	the correct output for every case, and the
	correct <code>tokens.txt</code> and <code>nametable.txt</code>
<code>README.txt</code>	how to run it

The manifest is `test_cases/cases.txt`. One line per case, tab separated: the case name, the exit status it must produce, its standard-output points, its file points, and the arguments it runs your program with. Every point value in Table 1 is read off that file, and so can yours be, before you write a line.

17.2 What Each Case Covers

This is a test suite for a piece of software, and every case is in it because it covers one scenario the scanner has to handle. `test_cases/purposes.txt` says in one line what that scenario is, for all one hundred eighty of them, and no two cases cover the same one.

Four lines from the purposes file

<code>o17_less_at_end_of_file</code>	the file ends in <code><</code> , so the lookahead has nothing to read and <code><</code> is a complete token
<code>c21_star_space_slash_does_not_close</code>	a star and a slash with a space between them inside a comment, which does not close it
<code>e36_nul_byte</code>	error 10 on a zero byte, which a scanner using C string functions treats as the end
<code>n09_seventy_identifiers</code>	seventy names, so a table starting at sixty-four has to grow

Read that file straight through before you start. It is the shortest complete list of the behavior this specification asks for, in the order the categories come in, and it is the fastest way to find the part of your scanner you have not thought about yet.

When a case fails, the autograder prints its line beside the failure, so the report says what was being checked rather than only that something did not match. The same line is the second line of `failed_cases/<case>/info.txt`.

17.3 How to Run It

```
unzip lex_test.zip
cd lex_test
ls lex.c
chmod +x test_lex.sh
./test_lex.sh lex.c
```

Your `lex.c` goes directly inside `lex_test`, beside `test_lex.sh`, and you run the script from inside that folder: it looks for `test_cases/` and `expected_results/` in the directory you run it from. Add `-quiet` to print the summary only.

Run it as often as you like. Nothing is recorded and nothing is submitted from here.

17.4 What the Script Does

1. Checks that your file is named `lex.c`, and refuses to go on if it is not.
2. Compiles it with the command in Section 15.
3. Runs every case, deleting `tokens.txt` and `nametable.txt` before each one, so that each case is judged on what it wrote itself.
4. For each case, compares the exit status and your standard output, and on a case that carries file points compares your two files as well.
5. Prints your score by category, and checks the one code requirement no case can see.

A case's standard-output points and its file points are separate. You can earn one and lose the other, in either direction, which is why the summary prints both figures before it prints your score.

17.5 Reading the Result

This is a real summary, from a scanner whose only fault is that it takes the shortest match on `<=`:

The summary from `./test_lex.sh lex.c -quiet`

```
=====
Test Summary
=====
Total Tests:      180
Passed:           174
Failed:           6

Standard output:  85.95 / 90
Written files:    8 / 10
HW2 program score: 93.95 / 100
```

Specified scanner behavior your program implements correctly: 94.0%
(This suite covers 100% of the scanner this handout specifies.
The figure above is how much of it your program gets right.)

By category:

Scanning basics	7.75 / 11
Whitespace and layout	9 / 9
Operators and longest match	17.2 / 20
Reserved words against identifiers	10 / 10
Comments	11 / 11
Name table	9 / 9
Lexical errors	25 / 25
Argument and file contract	5 / 5

Code requirements:

all clear (file name correct, compiles with the course flags, author set)

=====

FINAL SCORE: 93.95 / 100

=====

Cases that failed:

s02_every_token
s04_all_symbols
s06_spine_program
o04_relational_all
o09_operators_no_spaces
o15_every_pair_adjacent

One wrong decision cost six points, in two categories and in both streams at once. Six of the one hundred eighty cases hold a `<=`; each of them lost its output points, and the two of those that also carry a file point lost that as well, because `tokens.txt` carries the same extra token. Nothing else in that scanner is wrong, which is what the other seven categories say.

The script writes two directories, both deleted and rebuilt on every run:

`student_output/`

What your program printed, one file per case.

`failed_cases/`

One folder per failing case, holding `info.txt` with the command that was run, `diff.txt` with your output against the expected output, and `tokens.txt.diff` and `nametable.txt.diff` when the case lost its file points.

In every diff, lines beginning with `<` are yours and lines beginning with `>` are expected.

17.6 How Complete the Cases Are

The cases reach every branch of the reference implementation. That was measured rather than assumed: the reference was compiled with coverage instrumentation, every case was run against it, and every decision it makes was taken both ways. The only lines no case reaches are ones no source program can reach: what the reference does when memory runs out or a file cannot be written, and two guards against states the design has already ruled out.

Coverage is why some of the cases look strange. A file whose last line has no newline, a file large enough to make a growing buffer grow, a file whose line endings are carriage returns alone, a file with a carriage return in the middle of a line, a zero byte inside a declaration: each of those is in the bundle because measuring found behavior that no ordinary-looking program reached.

The cases also separate a correct scanner from a nearly correct one. Twenty-two deliberate faults were seeded into the reference one at a time, each one a mistake a submission can really contain, and the suite caught all twenty-two. That is what the weighting is for. A scanner that ignores a nested comment opener and is otherwise right scores 99. A scanner that takes the shortest match and is otherwise right scores 94. A scanner that never records where a name was first seen scores 14. Each number is what the suite actually gave that fault.

18 Submission Deadlines and Late Policy

All deadlines use U.S. Eastern Time (Orlando, FL). The Webcourses submission timestamp is authoritative. A submission is considered late if it is uploaded after the posted due date or time on Webcourses. Late submissions are accepted for up to 48 hours after the Webcourses due date, subject to the penalties in Table 6.

Late by	Penalty
0:00:01 to 12:00:00	−5 points
12:00:01 to 24:00:00	−10 points
24:00:01 to 36:00:00	−15 points
36:00:01 to 48:00:00	−20 points
After 48:00:00	not accepted; recorded as missed, 0 points

Table 6: The late penalty, looked up by how late the Webcourses timestamp is. Deductions are points off the homework’s maximum score, not a percentage, and are applied once, after grading.

Resubmissions after 48 hours are not accepted. Individual device or network problems do not justify exceptions: submit early and verify your upload.

19 Submission Instructions

Submit on **Webcourses** before the due date. Submissions by email, chat, cloud link, or any other channel are not accepted.

19.1 What to Submit

Tick each box at the upload screen. A conditional line says its condition first, so you can skip it when the condition is not yours.

- ☐ **lex.c**: your source code, named exactly that (Section 16.1), with **Author(s):** in the header comment filled in.
- ☐ **If you used any AI tool**: the AI Usage Disclosure Form **and** the conversation log, both of them. Section 2 says what goes in each.
- ☐ **If you used no AI tool**: nothing further.

That list is the whole submission. There is no PDF report, no screenshot, and no archive: the autograder run the instructor performs on your submitted file is the evidence. Uploading anything beyond the list costs nothing and is ignored.

Submit the file you tested. The instructor re-runs the autograder on the file you upload, and that run is the score. If the file you submit is not the file you tested on Eustis, your score follows the file you submitted. Before you upload, run `./test_lex.sh lex.c` one last time on the exact file and read the **FINAL SCORE** line.

20 Grading

Your program score is the autograder's final score, out of 100. There are no hidden tests, and what you see when you run the script on Eustis is what you get, provided your code compiles there. This section says exactly how that number is arrived at. It applies to you and to the teaching assistants alike.

20.1 How a Case Is Scored

Each case is scored on two things, separately.

Standard output

90 points across the one hundred eighty cases. A case earns its standard-output points when the exit status is what the manifest requires **and** your output matches the expected file. Both, or nothing: there is no partial credit inside a case.

The two files

10 points, one each on ten cases. Such a case earns its file point when `tokens.txt` and `nametable.txt` both exist under those names and both match. Both, or nothing. The

file point does not depend on the exit status, so a case can lose its output points and keep its file point.

Comparison ignores blank lines and any difference in spaces and tabs. It does not ignore where your line breaks fall, what the text says, or what order it comes in. The same rule applies to the two files.

A crash or a timeout scores nothing for that case. Each case is given five seconds.

These are never compared: which stream you printed to, because standard output and standard error are captured together; blank lines, indentation and spacing inside a line; compiler warnings, as Section 15 says; and code style beyond the header comment Section 16.2 requires.

20.2 Grading Order

Every submission is graded in the same order. A submission that fails at one point does not go on to the next, and nothing later restores what an earlier point took away.

1. **Similarity review**, as Section 3 describes it.
2. **The automatic-zero checks** of Section 20.3, all of which can be read off the submission before any test runs.
3. **The autograder**, run on Eustis on one submission at a time, with the same script, cases, expected outputs, and compiler commands you were given. The line it prints as `FINAL SCORE: nn.nn / 100` is your program score, and that is the number recorded in Webcourses.

20.3 Automatic Zero

Each line below replaces your score with 0. It is not a deduction and no later stage of grading restores it. **Read the list before you upload.** Every trigger on it is something you can check in under a minute, and every one of them has ended a submission whose program was completely correct.

Your homework scores 0 if any one of these is true:

- The submitted file is not named `lex.c`.
- It does not compile on Eustis with the published command.
- It is written in a language other than C.
- The required header comment is missing.
- The header comment's `Author(s):` line still reads the placeholder.
- Plagiarism.
- AI was used and no disclosure was filed.

The file name is in Section 16.1 and the compile command in Section 15. Plagiarism is in Section 3 and the disclosure rule in Section 2.

20.3.1 What Does Not Trigger One

- **Webcourses renaming your file when you resubmit**, inserting a number before the extension. That is the LMS renaming your file, not you. The rule is about the name you typed.
- **Compiler warnings**. See Section 15.
- **A late submission**, until the late window closes. Section 18 holds the schedule and the window.
- **An incomplete AI disclosure**. One of the two files missing is a deduction, not a 0. Only a disclosure that was never filed at all is the trigger above. Section 2 draws the line and states the cost.
- **A program that fails cases**. Failing cases is a score, not a trigger. Submit it.

20.4 AI Disclosure Requirements and the Late Penalty

Neither of these comes from the autograder. Both are applied afterwards, in Webcourses, to the number it printed.

- An incomplete AI disclosure, at the cost stated in Section 2.
- A late submission, at the cost in the schedule in Section 18.

20.5 What Grading Does Not Include

- **No curve, and no rounding up**. If the autograder says 87.50, the score is 87.50.
- **No discretionary adjustment** up or down for anything outside the rules in this section. Your program score comes from the autograder, and the autograder's cases and their point values are in your hands before you start.
- **The AI-disclosure deduction and the late penalty are applied afterwards**, in Webcourses, to the autograder's number: the deduction by the rule in Section 2, the penalty from the timestamp the LMS holds (Section 18).

21 Frequently Asked Questions

Every entry is a symptom in the words a student would use, and every entry appears in the table of contents, so you can find yours without reading the rest. Where the answer is a rule stated earlier, the entry says which section holds it.

21.1 Recognizing Tokens

21.1.1 Everything matches until partway through, and then every number in my token list is wrong

One token too many or too few was produced at that point, and everything after it shifted. Find the first line of `diff.txt` that differs and look at the lexeme there, not at the token list. The two usual causes are a two-character operator read as two tokens (Section 9) and a reserved word matched by prefix (Section 8).

21.1.2 My scanner reads `odd` as the reserved word `od`

Your lookup compares only as far as the keyword's length. Collect the whole run of letters and digits first, then compare the finished run against the whole spelling. Section 8 has the rule and Table 4 has the spellings.

21.1.3 Is `<>` one token or two?

Two: `<` then `>`. There is no `<>` in this language, and not-equal is `!=`. Table 3 is the whole symbol list.

21.1.4 I cannot keep `=`, `==` and `:=` straight

`==` compares and is code 7. `=` assigns and is code 18. `:=` gives a constant its value and is code 19. Table 5 shows how a scanner tells the first two apart.

21.1.5 My program reports error 8 on a line that multiplies and then divides

`b*/c` is a star immediately followed by a slash, which is error 8 wherever it appears outside a comment. Section 10 has the rule. Write a space between the two operators.

21.1.6 My comment looks closed but my program says error 7

Count the characters. In `/*` the slash is the second character of the opener, so the comment never closes. `* /` with a space, and a star ending one line with a slash beginning the next, do not close one either. Section 10.1 lists every shape.

21.1.7 My first comment works and my second one does not

You are setting a flag when a comment opens and not clearing it when it closes. A closed comment leaves the scanner outside a comment again. Section 10.1 says what that means for the case after it.

21.1.8 My program crashes on a file that ends in an operator

You looked two characters ahead when only one was there. A file may end in `<`, `>`, `=`, `/` or `*`, and each of those is a complete token; it may end in `!` or `:`, and each of those is an error. Section 9 has the rule.

21.2 Whitespace and Positions

21.2.1 My columns are all one too large after the first line

You are counting the carriage return as a column. Your file has CR LF line endings, which is normal, and Section 11 says a carriage return advances nothing.

21.2.2 Does a tab count as one column or eight?

One. Section 11 has the rule. Counting eight makes your columns agree with what your editor shows and disagree with every expected file.

21.2.3 Is `x < = y` the same as `x <= y`?

No. The two characters of a two-character operator have to be adjacent, so that is `<` and then `=`, four tokens in all. Section 11 has the rule, and it is an error rather than two tokens when the first character is `:` or `!`.

21.2.4 Where do I actually need a space between tokens?

In two places only: between a number and a letter after it, and between two words. Everywhere else the change of character class is enough. Section 11 works through both.

21.3 Errors

21.3.1 My error message is right but the case still fails

Check the exit status with `echo $?` after the run. An error must leave with a non-zero status. Section 14 has the rule.

21.3.2 My output has strange characters in it where the error message should be

You printed the offending byte instead of its value. Error 10 in Section 14 prints byte `0xHH` and never the byte itself.

21.3.3 My program stops early on a case whose file looks perfectly ordinary

Look at the bytes rather than at the text. One case holds a zero byte inside a line, and a scanner that treats the source as a C string ends there. Section 6 says to read to the end of the file.

21.3.4 My file has three bad characters. Should I report all three?

No. Report the first and stop. Section 14 says why, and one case exists to check it.

21.3.5 My program refuses a bad argument count, but the `t` cases still fail

The message text on those cases is compared. Section 5.1 gives both messages exactly; print them character for character.

21.4 Output and Files

21.4.1 My columns do not line up with the samples in this handout

That is fine. Spaces and tabs are ignored in the comparison. What matters is the text, its order, and where your line breaks fall. Section [20.1](#) states what is compared.

21.4.2 I lost the file point on every case that carries one

Either the files were not written, or their names are wrong. They are `tokens.txt` and `nametable.txt`, exactly, in the directory the program is run from. Section [13](#) gives both formats.

21.4.3 Should my program still write the two files when it finds an error?

Yes. They hold what was scanned before the error, the same as the printed sections. Section [13](#) states it.

21.4.4 The long cases stop partway through and the short ones all pass

Your buffers are a fixed size. One case is 5,770 bytes and produces 1,706 tokens and 100 distinct identifiers. Section [6](#) has the rule.

21.5 Form and Submission

21.5.1 There are one hundred eighty cases. Where do I start?

Read `test_cases/purposes.txt` straight through. Each line names one scenario, and together they are the whole behavior this handout specifies, which makes the list a checklist for your own scanner. Section [17.2](#) explains the file.

21.5.2 Why are there so many cases for one program?

Because a scanner has many small behaviors and each one can be wrong on its own. Every case covers a scenario no other case covers, so a failure tells you which behavior is missing rather than that the program is wrong somewhere. Section [17.2](#) is where that is written down.

21.5.3 Which file do I upload?

`lex.c`, and nothing else unless you used an AI tool. Section [19.1](#) is the checklist.

21.5.4 The autograder gave me one score and Webcourses shows another

The autograder's `FINAL SCORE` is your program score. An AI-disclosure deduction or a late penalty is applied afterwards, in Webcourses. Section [20.4](#) says which is which.

21.5.5 Something in this handout is unclear or appears wrong

Ask before you rely on your interpretation. See Section [1](#). A clarification issued by the instructor governs, and it is announced to everyone.

21.5.6 I cannot finish and the deadline is close

Submit what you have anyway, and email the instructor before the deadline. A submission that documents an honest failure is worth far more than a silent zero. What cannot be helped is a student who submits nothing and says nothing until the problem is urgent.